

Resumen global: Tarea 7 al 10

Guía Técnica para el Despliegue de Arquitectura Hadoop: Del Modelo a la Realidad

1. Modelos de Arquitectura: Pseudoclúster vs. Clúster Real

La estrategia de implementación se divide en dos fases prácticas principales, cada una con objetivos y configuraciones de red diferenciadas.

1.1 Práctica 4: Pseudoclúster (Entorno de Aprendizaje)

- **Definición:** Configuración de un solo nodo (Single Node) donde todos los demonios (`NameNode` , `DataNode` , `ResourceManager`) se ejecutan en una única máquina.
- **Objetivo:** Facilitar el aprendizaje y realizar pruebas de concepto rápidas.
- **Configuración Clave:** El parámetro `fs.defaultFS` apunta directamente a `localhost` .

1.2 Práctica 3: Clúster Real (Entorno de Producción)

- **Definición:** Arquitectura distribuida compuesta por un nodo Maestro y múltiples nodos Esclavos.
 - **Estructura Típica:**
 - **Maestro:** Ejecuta `NameNode` y `ResourceManager` .
 - **Esclavos:** Ejecutan `DataNode` y `NodeManager` .
 - **Objetivo:** Lograr alto rendimiento y simulación de entornos productivos.
 - **Configuración Clave:** Requiere una identidad de red única (DNS/IP) y comunicación fluida vía SSH.
-

2. Fase 1: Los Cimientos y Requisitos Previos

La estabilidad del clúster depende del cumplimiento estricto de una base técnica uniforme en todas las máquinas involucradas.

Requisito	Descripción
Software Base	Instalación de Java (JDK) en la misma ruta exacta en todos los nodos.
Usuario	Creación del usuario único <code>hadoop</code> con permisos idénticos.
Variables de Entorno	Configuración de <code>.bashrc</code> o <code>/etc/profile</code> con <code>JAVA_HOME</code> y <code>HADOOP_HOME</code> .
Estructura de Directorios	Creación de una carpeta central de datos (ej. <code>/datos</code>) con permisos de escritura asegurados.
Servicio SSH	Debe estar instalado y activo para la comunicación entre procesos.

3. Configuración del Núcleo (Single Node)

En el entorno de un solo nodo, la configuración se centra en definir el sistema de archivos y la ubicación de los metadatos.

Archivos de Configuración Esenciales:

- `core-site.xml` : Define el sistema de archivos por defecto.
 - `fs.defaultFS` : `hdfs://localhost:9000`
- `hdfs-site.xml` : Define las políticas de almacenamiento.
 - `dfs.replication` : Establecido en **1** (sin replicación en nodo único).
 - `dfs.namenode.name.dir` : Ruta hacia los datos del NameNode (ej. `/datos/namenode`).

Formateo y Verificación Inicial:

- **Crítico:** El comando `hdfs namenode -format` solo debe ejecutarse **una vez** antes del primer arranque. Su ejecución posterior borrará todos los datos.
 - **Arranque:** Se inicia con el comando `start-dfs.sh`.
 - **Verificación (JPS):** Se debe confirmar la ejecución de `NameNode`, `DataNode`, `SecondaryNameNode`, `ResourceManager` y `NodeManager`.
-

4. Fase 2 y 3: Expansión al Clúster Real (Multi-Node)

La transición a un clúster real implica la clonación del nodo base y la reconfiguración de la red y el rol del maestro.

4.1 Estrategia de Clonación

1. Utilizar la máquina del Pseudoclúster como imagen base.
2. Clonar la máquina para crear los nodos esclavos (ej. Nodo 2 y Nodo 3).
3. **Importante:** Tras la clonación, ejecutar `rm -rf /datos/*` en los esclavos para limpiar el ID del clúster anterior.

4.2 Identidad de Red y Resolución de Nombres

Cada nodo debe tener un nombre de host único establecido mediante `hostnamectl set-hostname [nombre]`. El archivo `/etc/hosts` debe ser idéntico en todos los nodos y listar las IPs y nombres de toda la infraestructura:

- `192.168.58.129 nodo1`
- `192.168.58.130 nodo2`
- `192.168.58.131 nodo3`
- **Advertencia:** Se deben eliminar las referencias a `127.0.1.1` asociadas a los nombres de los nodos.

4.3 Configuración de Confianza SSH (Passwordless)

El Maestro debe poder acceder a los Esclavos sin contraseña para lanzar procesos. El flujo es:

1. **Generar Claves:** `ssh-keygen -t rsa` en el Maestro.
 2. **Distribuir Autoridad:** `ssh-copy-id nodo[X]` hacia cada esclavo.
 3. **Verificación:** Probar el acceso directo `ssh nodo2`.
-

5. Reconfiguración del Maestro y Gestión de Recursos

Para que el Maestro reconozca la red distribuida, se deben actualizar sus archivos de configuración y definir la fuerza de trabajo.

5.1 Actualización de Archivos Core y HDFS

- `core-site.xml` : Cambiar `localhost` por el nombre del maestro (`nodo1`).
- `hdfs-site.xml` : Incrementar `dfs.replication` a `2` (para reflejar el uso de dos esclavos).

5.2 Definición de la Fuerza de Trabajo (Archivo `workers`)

Ubicado en `/opt/hadoop/etc/hadoop/workers` , este archivo debe listar exclusivamente a los esclavos (`nodo2` , `nodo3`).

- **Pro-Tip:** Eliminar `localhost` o `nodo1` de este archivo para asegurar que el Maestro solo gestione recursos y no procese datos directamente.

5.3 Configuración de YARN (`yarn-site.xml`)

Se debe indicar a los esclavos dónde reportar su capacidad de CPU y RAM:

- `yarn.resourcemanager.hostname` : `nodo1`
- `yarn.nodemanager.aux-services` : `mapreduce_shuffle`

6. Operación y Mantenimiento: La Regla de Oro

El despliegue de configuración debe seguir un principio de centralización:

Nunca editar configuraciones manualmente en los esclavos. Se debe configurar todo en el Maestro y distribuir mediante el comando `scp` :

```
$ scp -r /opt/hadoop/etc/hadoop nodo2:/opt/hadoop/etc/  
$ scp -r /opt/hadoop/etc/hadoop nodo3:/opt/hadoop/etc/
```

Secuencia de Arranque del Clúster

1. **Paso 0:** Limpieza de datos en esclavos (`rm -rf /datos/*`).
 2. **Paso 1:** Formateo (solo si es la primera vez).
 3. **Paso 2:** Arrancar HDFS (`start-dfs.sh`).
 4. **Paso 3:** Arrancar YARN (`start-yarn.sh`).
 5. **Paso 4:** Arrancar el historial (`mapred --daemon start historyserver`).
-

7. Verificación de Procesos e Inspección Final

Verificación por Nodo (Comando `jps`)

- **Nodo 1 (Maestro):** `ResourceManager` , `NameNode` , `SecondaryNameNode` .
- **Nodos 2 y 3 (Esclavos):** `NodeManager` , `DataNode` .

Monitoreo y Rendimiento

- **Reporte HDFS:** `hdfs dfsadmin -report` (Debe mostrar 2 datanodes vivos).
- **Interfaces Web:**
 - HDFS NameNode: `http://nodo1:9870`
 - YARN ResourceManager: `http://nodo1:8088`
- **Benchmarking:** Uso de `TestDFSIO` para medir el rendimiento del sistema de archivos (ej. `hadoop jar ... TestDFSIO -write -nrFiles 10 -fileSize 100`).

Guía para Pruebas de Concepto en Clusters Big Data

1. Preparación del Dataset y Carga en HDFS

Antes de iniciar el procesamiento, es imperativo poblar el ecosistema con datos de prueba. El procedimiento utiliza el archivo de patentes de Estados Unidos denominado `cite75_99.txt` .

- **Comando de Carga:** `hdfs dfs -put cite75_99.txt /practicass`
 - **Punto Crítico de Control:** Este paso valida la capacidad de escritura en el Sistema de Archivos Distribuido (HDFS). Si esta operación falla, no se debe proceder con las fases de MapReduce.
-

2. Nivel 1: Validación de Infraestructura (Shell de Linux)

Este nivel representa el enfoque de "menor fricción". Se utiliza la librería *Hadoop Streaming* para simular un programa MapReduce utilizando comandos

nativos de Unix (*pipes*), permitiendo validar la infraestructura sin necesidad de compilación compleja.

2.1. Objetivos y Herramientas

- **Mapper:** Se utiliza `cut -f 2 -d ,` para extraer el segundo campo (patente citada).
- **Reducer:** Se utiliza `uniq` para eliminar duplicados y obtener una lista única.
- **Resultado esperado:** Validación inmediata de la infraestructura y el flujo de E/S.

2.2. Sintaxis del Comando y Ejecución

La ejecución requiere definir rutas absolutas y parámetros específicos:

- **Librería:** Ruta al JAR de `hadoop-streaming` (ej. versión 2.9.0).
- **Directorio de salida:** El directorio de destino **no debe existir** previamente para evitar errores de ejecución.

2.3. Monitoreo en Tiempo Real (YARN)

La verificación se realiza a través de la interfaz web del Resource Manager (`http://nodo1:8088`).

Criterio de Éxito	Descripción
State	Debe marcarse como <code>SUCCEEDED</code> .
Maps Completed	El número debe coincidir con los bloques del fichero (ej. 2 maps).
Reduces Completed	Confirmación de la tarea de reducción (ej. 1 reduce).

2.4. Variación 1B: Job 'Map-Only'

Es posible optimizar el proceso desactivando el Reducer mediante el flag `-D mapred.reduce.tasks=0`.

- **Efecto:** El resultado refleja la fragmentación física de HDFS.
- **Salida:** Se genera un fichero de salida por cada bloque de datos procesado (ej. `part-00000` para el bloque 1 y `part-00001` para el bloque 2).

3. Nivel 2: Introducción de Lógica Externa (Python)

Este nivel verifica la capacidad del cluster para ejecutar intérpretes de lenguajes de scripting y distribuir archivos adicionales a los nodos de trabajo.

- **Lógica Aplicada:** Se sustituyen los comandos de Shell por un script Python (`rand.py`) que filtra aproximadamente el 10% de las filas de forma aleatoria.
 - **El Flag `file` :** Es un parámetro **crítico**. Envía el script físico a los nodos de trabajo mediante el *Distributed Cache*. Sin este flag, el Mapper fallará con un error de "file not found".
-

4. Nivel 3: MapReduce Nativo en Java

Considerado el estándar empresarial, este nivel valida el *toolchain* de compilación, el *classpath* y la capacidad completa de procesamiento distribuido (MapReduce V2).

4.1. Anatomía del Código (`MyJob.java`)

El desarrollo requiere implementar tres componentes clave en una clase Java:

1. **MapClass:** Divide la línea por comas e invierte el orden (Clave = patente citada, Valor = patente que cita).
2. **Reducer:** Realiza una concatenación de valores en una lista separada por comas para cada clave única.
3. **Driver:** Configura el Job, definiendo las clases Mapper, Reducer, tipos de clave de salida y formatos de entrada.

4.2. Proceso de Compilación y Empaquetado (Build)

El flujo para transformar el código fuente en un ejecutable distribuido es el siguiente:

1. **Configurar Classpath:** Exportar `HADOOP_CLASSPATH` para que Java localice las librerías de Hadoop.
2. **Compilar:** Usar `hadoop com.sun.tools.javac.Main MyJob.java` para generar archivos `.class`.
3. **Crear JAR:** Empaquetar todas las clases en un único archivo (ej. `MyJob.jar`).

4.3. Ejecución del JAR Nativo

El comando de ejecución sigue la estructura: `hadoop jar [artefacto.jar] [Clase Driver] [Input] [Output]`

5. Verificación Final de Resultados

Una vez finalizada la ejecución de nivel empresarial, se debe inspeccionar el contenido generado en HDFS mediante el comando: `hdfs dfs -cat /resultado7/part-r-00000 | head -5`

El formato de salida esperado presenta la **Patente Citada** (Clave) seguida de la lista concatenada de **Patentes que la citan** (Valores).

Lista de Validación de Operatividad

- **Infraestructura Base:** Verificada mediante Shell Streaming e I/O de YARN.
- **Entorno de Scripting:** Verificado mediante la distribución de ficheros Python.
- **Desarrollo Nativo:** Verificado mediante el proceso de compilación y ejecución de JARs en Java.

Conclusión: Una vez superados estos tres niveles, el cluster Hadoop se considera plenamente operativo y preparado para la implementación de analíticas complejas sobre datasets de gran escala.

Guía de Implementación y Gestión de Apache Hive: Manual de Big Data

1. Arquitectura de Apache Hive: El Puente de Datos

Hive opera bajo una arquitectura dual que permite a los analistas utilizar interfaces familiares mientras se aprovecha la potencia del procesamiento distribuido.

- **La Piel (User View):** Interfaz amigable basada en **SQL (HiveQL)** que permite definiciones lógicas de datos.
- **El Músculo (System View):** Motor de procesamiento basado en **MapReduce** y almacenamiento en **HDFS**.

- **Mecanismo de Traducción:** Hive actúa como el mediador que convierte las sentencias SQL en tareas de ejecución de bajo nivel.
-

2. Protocolo de Instalación y Configuración del Entorno

La implementación de Hive se divide en cuatro fases críticas que garantizan la estabilidad del sistema.

Fase 1: Preparación del Terreno (HDFS)

Es obligatorio configurar las rutas físicas en HDFS antes de la inicialización.

- **/tmp:** Espacio destinado a operaciones temporales.
- **/user/hive/warehouse:** Almacén físico predeterminado para los datos.
- **Acción Crítica:** Se debe aplicar el permiso `chmod g+w` (escritura para grupo) en ambas rutas. La omisión de este paso resultará en el fallo de la inicialización.

Fase 2: Instalación y Variables de Entorno

1. **Descomprimir:** El directorio objetivo estándar es `/opt/hadoop/hive`.
2. **Variables .bashrc:** Es necesario exportar `HIVE_HOME` y añadirlo al `PATH`.
3. **Enlace Hadoop:** Renombrar el archivo `hive-env.sh.template` a `hive-env.sh` y definir la ruta de `HADOOP_HOME`.

Fase 3: Configuración del Sitio (hive-site.xml)

Se debe crear el archivo `hive-site.xml` a partir de la plantilla `hive-default.xml.template`. Las propiedades críticas para evitar conflictos de escritura en entornos locales incluyen:

- `system:java.io.tmpdir`: Definir ruta temporal (ej. `/tmp/hive/java`).
- `system:user.name`: Configurar el nombre de usuario dinámico (`${user.name}`).

Fase 4: Inicialización del Metastore

El **Metastore** actúa como la "memoria" de Hive (utilizando Derby DB por defecto) y debe inicializarse antes del primer uso mediante el comando:

```
schematool -dbType derby -initSchema
```

3. Ingeniería de Datos y Estructuras Complejas

Hive permite la manipulación de datasets con estructuras anidadas y tipos de datos complejos, superando las limitaciones de las tablas planas tradicionales.

Anatomía de un Dataset Complejo

Para un archivo de texto (ej. `empleados.txt`), se definen delimitadores específicos:

- **PIPE (`|`)**: Separador de columnas.
- **COMA (`,`)**: Separador de Arrays (colecciones).
- **DOS PUNTOS (`:`)**: Separador de Maps (Clave:Valor).

Tipos de Datos y Consultas Avanzadas

Estructura	Descripción	Ejemplo de Consulta (HiveQL)
ARRAY	Lista de elementos	<code>SELECT work_place[0] FROM empleados_internal;</code>
STRUCT	Objeto con campos definidos	<code>SELECT sex_age.age FROM empleados_internal;</code>
MAP	Pares Clave-Valor	<code>SELECT skills_score['DB'] FROM empleados_internal;</code>

4. Gestión de Tablas: Internas vs. Externas

La elección del tipo de tabla es fundamental para el ciclo de vida de los datos y la seguridad de la información.

Tablas Internas (Managed Tables)

- **Definición**: Hive gestiona tanto los metadatos como los datos físicos.
- **Riesgo**: Al ejecutar `DROP TABLE`, se produce una **destrucción total**: se borran los metadatos y los archivos en HDFS.

Tablas Externas (Enterprise Standard)

- **Concepto:** Los datos existen independientemente de Hive. La plataforma solo los "lee".
 - **Seguridad:** Al ejecutar `DROP TABLE`, los metadatos se borran pero los **datos en HDFS permanecen intactos**.
 - **Sintaxis:** Requiere la palabra clave `EXTERNAL` y la cláusula `LOCATION` para apuntar a la ruta preexistente en HDFS.
-

5. Resolución de Problemas y Comandos Esenciales

Problemas Comunes y Soluciones

- **Warnings SLF4J:** Mensajes sobre múltiples bindings de librerías. **Acción:** Ignorar, son inofensivos.
- **Error de Metastore:** Hive no arranca o falla al crear tablas. **Solución:** Borrar el directorio `metastore_db` y ejecutar nuevamente `schematool -initSchema`.
- **Permission Denied:** Errores al escribir en `/tmp` o `/warehouse`. **Solución:** Verificar el permiso `chmod g+w` en HDFS.

Cheatsheet: Comandos Imprescindibles

Categoría	Comando
Sistema	<code>schematool -dbType derby -initSchema</code>
Sistema	<code>hive (shell)</code>
HDFS	<code>hdfs dfs -mkdir -p /ruta</code>
HDFS	<code>hdfs dfs -put local remoto</code>
HiveQL (DDL)	<code>CREATE EXTERNAL TABLE ... LOCATION ...;</code>
HiveQL (DML)	<code>LOAD DATA LOCAL INPATH ... INTO TABLE ...;</code>
HiveQL (DML)	<code>INSERT INTO ... VALUES ...;</code> (Lanza un Job de MapReduce)

BIG DATA EN LA PRÁCTICA: DE LA INFRAESTRUCTURA A LA ESTRATEGIA DE NEGOCIO

I. La Ruta del Operador de Datos: Evolución de Competencias

La metodología propuesta se basa en una progresión lógica de habilidades y responsabilidades técnicas, estructurada en tres fases principales:

Fase	Rol Asociado	Enfoque Principal	Herramientas Clave
01. Conexión	SysAdmin	Infraestructura y acceso remoto.	HiveServer2, Beeline.
02. Ciclo de Vida	Data Engineer	Ingesta, procesamiento y flujo estándar.	ETL, Analytics (NASA), MapReduce.
03. Proyecto Final	Data Analyst	Estrategia de negocio e insights.	Scouting, ROI, Business Value.

II. Infraestructura y Conectividad Remota: HiveServer2

La arquitectura de producción exige que el trabajo no se realice directamente en el nodo maestro. Para ello, se implementa **HiveServer2**, un servidor basado en **Apache Thrift** que habilita conexiones remotas.

Configuración de Seguridad y Acceso

Para facilitar la operatividad en entornos de desarrollo y permitir la suplantación de identidad (*impersonation*) sin la complejidad de una autenticación Kerberos, se requiere la modificación del archivo

```
/opt/hadoop/hive/conf/hive-site.xml .
```

- **Propiedad Crítica:** `hive.server2.enable.doAs`
- **Valor:** `false`

Protocolo de Conexión (Beeline)

La conexión se establece desde un cliente local (Nodo 3) hacia el servidor (Nodo 1) utilizando el cliente ligero de línea de comandos **Beeline**.

- **Protocolo:** JDBC sobre el puerto 10000.
- **Comando de conexión:** `!connect jdbc:hive2://nodo1:10000/ejemplo`

- **Recomendación técnica:** Copiar la configuración mediante `scp` del Nodo 1 al Nodo 3 para evitar la reconfiguración completa del software cliente.
-

III. Ciclo de Vida del Dato: Caso de Estudio Global (NASA)

El análisis de ~9,500 registros históricos de deslizamientos de tierra sirve para demostrar el ciclo completo del dato: Ingesta → Proceso → Visualización.

1. Ingesta y Analytics (MapReduce)

Hive actúa como una capa de abstracción que traduce sentencias SQL a tareas de MapReduce de forma transparente. El proceso incluye:

- Carga de archivos CSV locales a esquemas de tablas definidos.
- Ejecución de consultas SQL para filtros de catástrofes (ej. incidentes con >100 víctimas).
- Generación de rankings por países con mayor siniestralidad.

2. Enriquecimiento del Dato (Joins)

Para aportar contexto geográfico, se realiza una operación de `JOIN` entre la tabla de deslizamientos y una tabla de países, utilizando el nombre del país como llave de unión (`ON d.country_name = p.nombre`).

3. Exportación y Visualización

El ciclo se cierra permitiendo que el cliente visualice los resultados procesados por el cluster.

1. **Generación:** Hive genera un CSV mediante `INSERT OVERWRITE LOCAL DIRECTORY`.
 2. **Transferencia:** Los archivos se descargan a la máquina local vía FTP (FileZilla).
 3. **Visualización:** Se utiliza Excel para la limpieza de cabeceras y creación de gráficos.
-

IV. Proyecto Final: Operación 'Sucesor de Kroos'

Este caso de estudio aplica el Big Data al ámbito del fútbol profesional, con la misión de encontrar un relevo generacional para el mediocampo del Real Madrid optimizando la relación calidad/precio.

Ingeniería de Datos y ETL

Se utilizan fuentes de rendimiento deportivo (**FBref**) y datos financieros (**Transfermarkt**). Mediante **Python**, se realiza un pre-procesamiento para:

- Eliminar cabeceras repetitivas y limpiar el formato de edad.
- Extraer valores numéricos de campos complejos como `market_value_in_eur`.

Lógica de Negocio: El Filtro Estratégico

Se aplica un embudo de conversión para identificar a los "Top Prospectos":

1. **Candidatos Mundiales.**
2. **Estatus Galáctico:** Valor de mercado $\geq 50M$ €.
3. **Posición:** Mediocampistas (MF), excluyendo delanteros.
4. **Regla 'Adam Wharton':** Precisión de pases $> 75\%$.
5. **Juventud:** Edad ≤ 24 años.

Métricas de Evaluación

El éxito del análisis se mide mediante dos indicadores clave:

- **Impacto Total:** Suma de Pases Progresivos, Pases Clave y Creación de Tiros.
- **ROI (Rentabilidad):** Impacto Total dividido por el Precio (en millones).

V. Resultados: Informe de Scouting y Conclusión

El análisis identifica tres arquetipos de jugadores basados en las métricas de Hive:

Arquetipo	Jugador Ejemplo	Hallazgo Clave
El Rey	Pedri	Impacto máximo (595.0). El costo no es la prioridad.

La Ganga	Damián Rodríguez	ROI excepcional (228.0) con un coste reducido (500k).
El Equilibrio	Enzo Fernández	Impacto alto (436.0) y ROI moderado (5.12). Opción sensata.